Troubleshooting Microsoft Entra Connect Cloud Sync — Users Skipped / Not Provisioned to Entra (JoinNotFound, Scoping, Config & Quarantine)

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Troubleshooting Microsoft Entra Connect Cloud Sync (Users Skipped / JoinNotFound) 4

What you get . . . 4

What this delivers . . . 4

The diagnostic model . . . 4

Outcome . . . 5

Solution Overview 5

What this toolkit is . . . 5

Cloud sync is NOT Connect Sync — this matters . . . 5

What "JoinNotFound / skipped" actually means . . . 5

A subtlety that trips people up — provision on demand ignores scope . . . 6

The conditions this toolkit isolates . . . 6

The named artifacts this toolkit refers to . . . 7

The five diagnostic tools . . . 7

Where the vendor boundary sits . . . 8

How to use the rest of this toolkit . . . 8

Requirements 8

R1. Access to the provisioning-agent host server . . . 9

R2. Local administrator rights on the agent host . . . 9

R3. The AADCloudSyncTools PowerShell module (on the agent host) . . . 10

R4. Microsoft Graph read access (for the tenant-side scripts) . . . 10

R5. Entra admin center read access (Hybrid Identity Administrator or Global Reader) . . . 10

R6. The AD object identifier and the intended scope . . . 11

R7. On-premises Active Directory read access (for the object test) . . . 11

Checklist . . . 11

Global Requirements — Entra ID Solutions (category-level) 12

G1. Microsoft Entra ID tenant . . . 12

G2. Entra ID licensing (P1, P2 where noted) . . . 12

G3. Administrative roles . . . 12

G4. Break-glass (emergency access) accounts . . . 12

G5. Healthy on-premises Active Directory . . . 12

G6. An Entra Connect host server . . . 13

G7. UPN / domain hygiene . . . 13

G8. Network access to Microsoft cloud endpoints . . . 13

Diagnostic Workflow 13

Phase 1 — Collect (always start here) . . . 14
Phase 2 — Analyze (run the focused check for the flagged area) . . . 14
Phase 3 — Remediate (in the owning plane) . . . 16
Phase 4 — Verify (re-run until the object provisions) . . . 17
Testing — Confirming the Toolkit and the Fix 17
Test matrix overview . . . 18
T1 — Collector runs and writes a bundle . . . 18
T2 — Agent-health check is accurate . . . 19
T3 — Provisioning-log read returns the authoritative reason . . . 19
T4 — Job / SP / org check is accurate . . . 19
T5 — Object test classifies correctly . . . 19
T6 — Known-good object reads as provisioning (no false positives) . . . 19
T7 — Known-bad object reads as skipped (no false negatives) . . . 19
T8 — Post-fix re-run provisions the user (the decisive test) . . . 20
Overall exit criteria . . . 20
Troubleshooting — Cloud Sync "Users Skipped / JoinNotFound / Not Provisioned" Symptom → Cause → Fix Matrix 20
§The core symptom — provision on demand shows "User was skipped, JoinNotFound" . . . 20
§Nothing is being created at all (fleet-wide, not one user) . . . 21
§Agent health — the provisioning agent isn't working . . . 22
§Scope & matching — the object is not a create candidate . . . 23
§Already-exists — the "prior sync method" trap . . . 24
§Coexistence & thresholds — Connect Sync and cloud sync together . . . 24
§Reading the authoritative reason — provisioning logs . . . 24
§Script / tooling issues . . . 25
When to stop diagnosing and escalate to Microsoft . . . 25
7. Best Practices — Cloud Sync Provisioning Triage 25
8. Deliverables Summary — Troubleshooting Microsoft Entra Connect Cloud Sync (JoinNotFound) 27
Documentation . . . 27
PowerShell diagnostic tools (all read-only) . . . 27
What the buyer receives . . . 28
Outcome . . . 28
Appendix A — PowerShell Scripts 29

Executive Summary — Troubleshooting Microsoft Entra Connect Cloud Sync (Users Skipped / JoinNotFound)

Entra ID Solutions · Cloud Sync Diagnostics

You set up **Microsoft Entra Connect cloud sync** to synchronize on-premises Active Directory users into Microsoft Entra ID. It looks configured, the agent looks healthy — but **no accounts are being created**. When you run **provision on demand** you see **Import ✓ / In scope ✓ / Match ✓**, and then **Evaluate action** → **"User was skipped", SkipReason: JoinNotFound**. Nothing is wrong with the network; one specific condition is stopping the create — and cloud sync's own logs will tell you which, if you know where to read them.

This is a **diagnostics toolkit**, not a deployment guide. It gives an administrator a structured way to find *why* cloud sync is skipping users — a **read-only** workflow and five PowerShell tools that snapshot the agent, read the authoritative per-object reason from the provisioning logs, and classify the object — so the fix is named in minutes instead of guessed at over days.

What you get

• A plain-language **Solution Overview**: how cloud sync differs from the older Entra Connect *Sync*,

what **JoinNotFound** actually means (the evaluate step found no target to join and no new object to create), and the specific conditions that cause it — an already-existing target from a prior sync method, an out-of-scope OU, a disabled configuration, a quarantined job, a deletion threshold, a missing service principal, or an unhealthy agent.

• A **Requirements Guide** with verify / implement-if-missing for every read-access prerequisite.

• A **Diagnostic Workflow** — the ordered **Collect** → **Analyze** → **Remediate** → **Verify** process with a

symptom → tool map and explicit gates.

• **Five read-only PowerShell tools**: an agent-host collector, an agent-health check, a provisioning-

log reader (the authoritative skip reason by `ObjectGuid`/UPN), a synchronization-job/quarantine/SP check, and an Active Directory object test (critical-object, in-scope, UPN, duplicate).

• **Testing and Troubleshooting** guides — the decisive test is that, after the fix, provision on

demand creates the user and the provisioning log shows **action = Create**.

What this delivers

A named root cause for "cloud sync isn't provisioning users," reached with tools that change nothing on the agent, the tenant, the configuration, or the object — and a clear, owner-plane remediation for each condition.

The diagnostic model

Collect → Analyze → Remediate → Verify — an adaptation of the category's Assess → Configure → Pilot → Validate → Expand model to a read-only troubleshooting shape.

Outcome

The reason users are skipped — JoinNotFound and the condition behind it — isolated fast, so provision on demand creates the user, the provisioning log shows **action = Create**, and the account appears in Entra ID.

Solution Overview

What this toolkit is

This is a **diagnostics toolkit**, not a deployment guide. It assumes you already run — or are supposed to run — **Microsoft Entra Connect cloud sync**: the lightweight **provisioning agent** that synchronizes users, groups, and contacts from on-premises **Active Directory (AD)** into **Microsoft Entra ID**. Something in that flow has stopped working: the configuration looks complete, the agent shows healthy, but **no accounts are being created in Entra ID**. When you run **provision on demand** for a user you see **Import ✓ / In scope ✓ / Match ✓**, and then the **Evaluate action** step reports **"User was skipped"** with **SkipReason: JoinNotFound** — *"not a newly discovered entry to be provisioned … nor one with an update to flow to a previously matched target."* This toolkit collects the agent and tenant state, reads the authoritative per-object reason, and points you at the fix.

The core idea: **a cloud sync "users not provisioning" failure is almost never "cloud sync is broken."** It is one specific condition — the target already exists from a previous sync method, the user's OU is outside the configuration's scoping filters, the configuration is not enabled, the job is quarantined, an object-deletion threshold is blocking export, or a service principal is missing — producing a symptom (**JoinNotFound / skipped / nothing created**) that looks like a total failure. If you can name the condition, you can name the fix. The whole toolkit exists to help you name it fast.

Cloud sync is NOT Connect Sync — this matters

Microsoft ships **two** on-premises-to-Entra synchronization engines, and they fail for different reasons:

- **Microsoft Entra Connect *cloud sync*** — a **lightweight provisioning agent** installed on a

domain-joined server; the **cloud service does the heavy lifting**. Configuration lives in the Entra admin center. This is what the scenario above uses, and it is what **this toolkit diagnoses**.

- **Microsoft Entra Connect *Sync*** (the older, full "Azure AD Connect" client) — a full

synchronization engine (the `ADSync` service, the Synchronization Service Manager `miisclient.exe`) running on a dedicated server. Different tooling, different logs, different failure modes.

A common trap: an engineer who **last set this up years ago with Connect Sync** applies Connect Sync habits (the sync scheduler, `Start-ADSyncSyncCycle`, the sync rules editor) to a **cloud sync** configuration where none of that applies. Cloud sync is driven by the **provisioning service** and its **synchronization job**, read in the **provisioning logs** — not the ADSync engine. This toolkit keeps you on the correct engine.

What "JoinNotFound / skipped" actually means

When cloud sync evaluates a source (AD) object, it runs an ordered pipeline the **provision on demand** screen shows step by step:

1. **Import** — the object was read from AD.
2. **Determine if in scope** — does the object match the configuration's **scoping filters** (the

organizational units the configuration is set to synchronize)? *Note: provision on demand **ignores** scoping filters (see below); the scheduled cycle does not.*

1. **Match user** — does a **target** object already exist in Entra ID to **join** to (a soft/hard

match on `userPrincipalName` / `mail` / anchor), or is this a **new** object to **create**?

1. **Determine object action** (Evaluate) — decide **Create**, **Update**, or **Delete**.
2. **Provision** — perform the action against the target.

JoinNotFound is a **skip at step 4**: the engine found **no existing target to join** *and* the object is **not, at that moment, a valid new-provision candidate** — so there is nothing to create and nothing to update, and the object is **skipped**. Read literally, it means *"I have nothing to do with this object."* The job of diagnosis is to find out **why** it is neither a join nor a create — almost always one of the named conditions below.

A subtlety that trips people up — provision on demand ignores scope

Provision on demand runs the pipeline **for the one object you pick, ignoring the configuration's scoping filters** — so you can run it on a user who is *outside* the configured OUs. That has two consequences you must hold in mind:

• If **provision on demand skips** an object, the cause is usually **not** scope *for that on-demand

run* — look at **match/already-exists**, critical-object exclusion, or a stalled job.

• The **scheduled cycle only creates in-scope users.** So an object can provision fine on demand yet

never get created by the recurring cycle because its **OU is not in the scoping filters**. Scope is diagnosed against the **scheduled** behaviour, not the on-demand run.

The conditions this toolkit isolates

1. **The target already exists in Entra ID.** A matching object — commonly a **leftover from a prior

sync method **(e.g. an earlier Connect Sync deployment) or a** cloud-only** account with the same `userPrincipalName` / `proxyAddresses` — makes the evaluate step a soft-match/no-op, so nothing is created.

1. **The configuration is not enabled, or the user's OU is not in the scoping filters.** The scheduled

cycle will not create out-of-scope users. After changing scope or the configuration, you must **restart provisioning** to clear watermarks so objects are re-discovered.

1. **Excluded-by-design objects.** `IsCriticalSystemObject` (built-in accounts and groups) and

replication-victim objects are filtered out by default — testing with one of those looks like a failure but is expected.

1. **The job is quarantined.** If calls to the target consistently fail (for example bad admin

credentials), the synchronization job is **quarantined** and nothing exports until you clear it.

1. **An object-deletion threshold is blocking changes.** Where Connect Sync and cloud sync **coexist**

(both exporting to one tenant), the accidental-deletion-prevention threshold can block the run.

1. **A missing service principal —** `AzureDirectoryServiceAuthorizationFailed`**.** If the **"Microsoft

Entra AD Synchronization Service"** service principal is absent, the agent cannot authorize; sync must be re-enabled to recreate it.

1. **Agent health / connectivity.** The **"Microsoft Entra Provisioning Agent"** and **"Microsoft

Entra Connect Agent Updater" **services must be** Running**, the agent must show** Healthy **in the portal, and outbound** 80/443** to the documented endpoints must be open.

The named artifacts this toolkit refers to

Diagnosis is comparison, so the toolkit reuses **exact names** for the objects you supply or inspect. Reuse these verbatim (they appear in the scripts and the troubleshooting matrix); the en-dash is –:

Artifact	Name
The cloud sync configuration	`Cloud Sync – AD to Entra – Configuration`
An organizational unit that IS in the scoping filters	`Cloud Sync – In-Scope Users – OU`
Pilot / test scope OU	`Cloud Sync – Pilot Scope – Pilot`
The domain-joined server running the agent	`Cloud Sync – Provisioning Agent Host`
The timestamped diagnostics output folder	`Cloud Sync – Diagnostics – <timestamp>`

Microsoft's own fixed names are reused verbatim too: the **"Microsoft Entra Provisioning Agent"** and **"Microsoft Entra Connect Agent Updater"** Windows services, and the **"Microsoft Entra AD Synchronization Service"** service principal.

The five diagnostic tools

All five are **read-only**. They observe; they never remediate.

• `CLDSYNCDX-Collect-Diagnostics.ps1` — the entry point, run **on the provisioning-agent host**. A

single on-device collector that snapshots agent-service state, PowerShell execution policy, and proxy configuration, then (if the **AADCloudSyncTools** module is present) runs `Export-AADCloudSyncToolsLogs` and gathers the agent **Trace** folder, zipping everything to a timestamped output folder.

• `CLDSYNCDX-Check-Agent.ps1` — focused **agent-health** checks on the host (services Running,

execution policy sane, outbound 443 reachability, config/proxy present) with **PASS/FAIL** per precondition and the exact remedy for each FAIL.

• `CLDSYNCDX-Get-ProvisioningLog.ps1` — a Microsoft Graph read-only query of the **provisioning

logs **for a given AD** ObjectGuid **or** UPN**, printing the latest entries with the** action**,** status**, and** skipReason **(e.g.** `JoinNotFound`**) plus the per-step detail — the** authoritative** per-object reason.

• `CLDSYNCDX-Check-Provisioning.ps1` — a Microsoft Graph read-only check of the cloud sync

synchronization job (Active / Enabled / **Quarantined** with reason), whether the **"Microsoft Entra AD Synchronization Service"** service principal exists, and the org's `onPremisesSyncEnabled` — flagging `AzureDirectoryServiceAuthorizationFailed` and object-deletion-threshold symptoms.

• `CLDSYNCDX-Test-Object.ps1` — a local **Active Directory** read-only check for one user:

`IsCriticalSystemObject`, whether its OU is under the operator-supplied in-scope OU DNs, whether its UPN suffix is a routable/verified domain, and an advisory on a possible same-UPN/`proxyAddresses` object — to catch the excluded / out-of-scope / already-exists conditions.

Where the vendor boundary sits

This toolkit authors the **AD, agent-host, Entra, and Microsoft Graph** side at exact, click-by-click detail — that is where your read-only scripts and portal checks live. **Microsoft product internals — the cloud sync provisioning service pipeline, the exact wording of provisioning-log reason codes, and the AADCloudSyncTools cmdlet internals — are deferred to Microsoft's official documentation.** The toolkit tells you *what* the agent, the AD object, and the configuration must present; the remediation steps live in the owning plane.

How to use the rest of this toolkit

• `02-requirements.md` — the read access and vantage points to have ready before you diagnose.

• `03-implementation.md` **(Diagnostic Workflow)** — the ordered **Collect → Analyze → Remediate →

Verify **workflow and the** symptom → tool** map. Start here during an incident.

• `05-testing.md` — pass/fail gates that confirm the toolkit runs correctly and that a fix actually

cleared the failing condition.

• `06-troubleshooting.md` — the comprehensive **symptom** → **likely cause** → **fix** matrix for "users

skipped / JoinNotFound / not provisioned." This is the heart of the product.

• `07-best-practices.md` — repeatable triage habits, evidence collection, and clean escalation.

Standing constraint for the whole toolkit: every script and check here is **read-only**. Diagnostics observe; they never change the AD object, the tenant, the configuration, or the agent. You decide the fix; the toolkit only tells you where it is. The workflow adapts the category-standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model to a diagnostic shape: **Collect** → **Analyze** → **Remediate** → **Verify.**

Requirements

Before you can diagnose why **Microsoft Entra Connect cloud sync** is skipping users (JoinNotFound / not provisioned), you need read access to each place that holds evidence: the **provisioning-agent host**, on-premises **Active Directory**, and the **Entra ID tenant** (portal + Microsoft Graph). This section lists what to have ready. Each item follows **Description / Why / Verify / Implement if missing**. Category-wide

prerequisites are in `global-requirements-entra-id-solutions.md`.

Gather these *before* an incident where possible — assembling access mid-incident wastes time. Everything here is **read-only**; you never need write/remediation rights to run this toolkit.

> *Naming convention used throughout this toolkit (reuse these names verbatim; the en-dash is –): - The cloud sync configuration:* `Cloud Sync – AD to Entra – Configuration` *- An organizational unit that IS in the scoping filters:* `Cloud Sync – In-Scope Users – OU` *- Pilot / test scope OU:* `Cloud Sync – Pilot Scope – Pilot` *- The domain-joined agent server:* `Cloud Sync – Provisioning Agent Host` *- Microsoft's fixed names, reused verbatim: the* ***"Microsoft Entra Provisioning Agent"*** *and* ***"Microsoft Entra Connect Agent Updater"*** *services, and the* ***"Microsoft Entra AD Synchronization Service"*** *service principal.*
>
> ***Government / sovereign-cloud note:*** *if the tenant is* ***GCC High / DoD****, the admin portals differ (*`https://entra.microsoft.us`*, sign-in via* `https://login.microsoftonline.us`*) and the Microsoft Graph endpoint differs (*`https://graph.microsoft.us`*); feature/endpoint parity can lag Commercial. The agent-host and AD checks are identical, but* ***verify the portal, Graph, and object availability for GCC High*** *in your sovereign cloud before acting on the tenant-side checks.*

R1. Access to the provisioning-agent host server

Description: Local (or remote-management) access to the domain-joined Windows server where the **Microsoft Entra provisioning agent** for cloud sync is installed — enough to read its services, the agent **Trace** folder, the execution policy, and the proxy configuration.

Why: Agent-side truth the cloud cannot see lives here: whether the **"Microsoft Entra Provisioning Agent"** and **"Microsoft Entra Connect Agent Updater"** services are actually **Running**, and what the agent trace logs recorded. The scripts `CLDSYNCDX-Collect-Diagnostics.ps1` and `CLDSYNCDX-Check-Agent.ps1` run **on this host**.

Verify: You can open a PowerShell session on the host and run `Get-Service "*Provisioning Agent*","*Agent Updater*"` and see output, and browse `C:\ProgramData\Microsoft\Azure AD Connect Provisioning Agent\Trace`.

Implement if missing: Arrange local admin access, an RDP/remote-PowerShell session, or have an on-site operator run the read-only collector and return the zip. No changes are made to the host.

R2. Local administrator rights on the agent host

Description: Local administrator rights on the agent host, to read the agent services, the Trace folder, and to run `Export-AADCloudSyncToolsLogs`.

Why: The agent installs under `ProgramData` and runs as a service; a standard user cannot reliably read the Trace folder or export the diagnostic logs, so a check would report false negatives.

Verify: In an elevated PowerShell session, `Get-ChildItem "C:\ProgramData\Microsoft\Azure AD Connect Provisioning Agent\Trace"` returns files.

Implement if missing: Run the host-side scripts **elevated** (Run as administrator). The scripts read only; elevation is for visibility, not change.

R3. The AADCloudSyncTools PowerShell module (on the agent host)

Description: The **AADCloudSyncTools** module available on the agent host, which ships with the provisioning agent and wraps the log export and repair cmdlets (`Connect-AADCloudSyncTools`, `Export-AADCloudSyncToolsLogs`, `Repair-AADCloudSyncToolsAccount`, `Disable-AADCloudSyncToolsDirSyncAccidentalDeletionPrevention`).

Why: `CLDSYNCDX-Collect-Diagnostics.ps1` uses `Export-AADCloudSyncToolsLogs` (when present) to gather a complete agent log bundle. The module is also where the *remediation* cmdlets live (used in Phase 3, outside this read-only toolkit).

Verify: On the host, `Get-Module AADCloudSyncTools -ListAvailable` returns the module (it installs with the agent under the agent's PowerShell module path). If absent, the collector still gathers the services state and the Trace folder.

Implement if missing: Install it per Microsoft's guidance — `Install-Module -Name AADCloudSyncTools` (it depends on the Microsoft Graph and MSAL modules). Then `Connect-AADCloudSyncTools` to authenticate. Exact module setup is **deferred to Microsoft documentation**.

R4. Microsoft Graph read access (for the tenant-side scripts)

Description: The ability to run `CLDSYNCDX-Get-ProvisioningLog.ps1` and `CLDSYNCDX-Check-Provisioning.ps1` against Microsoft Graph with the read-only scopes `AuditLog.Read.All`, `Directory.Read.All`, and `Synchronization.Read.All` (the last for reading the synchronization job/quarantine state).

Why: The **provisioning logs** (the authoritative per-object skip reason) and the cloud sync **synchronization job** (Active / quarantined) are read through Graph — the same data you can click through in the portal.

Verify: Run `Connect-MgGraph -Scopes "AuditLog.Read.All","Directory.Read.All","Synchronization.Read.All"` and confirm consent succeeds; `Get-MgContext` shows the scopes granted.

Implement if missing: Install the Microsoft Graph PowerShell SDK (`Install-Module Microsoft.Graph -Scope CurrentUser`). Consent to the delegated **read** scopes above at first connect — no write consent is required or requested.

R5. Entra admin center read access (Hybrid Identity Administrator or Global Reader)

Description: Sign-in to the Microsoft Entra admin center (`https://entra.microsoft.com`; GCC High: `https://entra.microsoft.us`) with a role that can view **Entra Connect** → **Cloud sync** — the configuration, its scoping filters, the agent health, and the **provisioning logs**.

Why: Whether the configuration is **Enabled**, which **OUs are in the scoping filters**, whether the agent shows **Healthy**, and the per-object provisioning-log entries are all visible here. The **Hybrid Identity Administrator** role is the least-privileged role that can manage cloud sync; **Global Reader** gives

tenant-wide read.

Verify: Browse to **Entra ID** → **Entra Connect** → **Cloud sync** and confirm you can open the configuration `Cloud Sync – AD to Entra – Configuration`, read its scope, and open **Logs**.

Implement if missing: In Entra → **Roles & admins**, assign **Global Reader** (tenant-wide read, no change rights) or **Hybrid Identity Administrator**. Prefer read-only for diagnosis.

R6. The AD object identifier and the intended scope

Description: For the user that is not provisioning, its **AD** `ObjectGuid` (and/or its `UPN`), plus a recorded statement of which **OUs are supposed to be in scope** for `Cloud Sync – AD to Entra – Configuration` (the DNs of the in-scope organizational units).

Why: Diagnosis is comparison. The **provisioning logs are searchable by the AD** `ObjectGuid` — it is the reliable key to the per-object reason. Knowing the intended in-scope OUs turns "is this user in scope?" into a one-line DN match (`CLDSYNCDX-Test-Object.ps1` takes the in-scope OU DNs as a parameter).

Verify: You can state, for the failing user: its `ObjectGuid` (from `Get-ADUser <name> -Properties ObjectGuid`), its `UPN`, and the list of in-scope OU DNs the configuration is set to synchronize.

Implement if missing: Read the `ObjectGuid` on the agent host (or any domain member) with the ActiveDirectory module; read the configuration's scoping filters in **Entra Connect** → **Cloud sync** → the configuration → **Scope provisioning by … / Configure attribute mapping**.

R7. On-premises Active Directory read access (for the object test)

Description: Read access to Active Directory (the ActiveDirectory PowerShell module on the agent host or a domain member) to inspect the failing user's `IsCriticalSystemObject`, `distinguishedName`/OU, `userPrincipalName`, and `proxyAddresses`.

Why: The **excluded-by-design** (`IsCriticalSystemObject`), **out-of-scope OU**, and **already-exists / duplicate UPN or proxyAddress** conditions are AD-side facts. `CLDSYNCDX-Test-Object.ps1` reads them read-only.

Verify: `Get-ADUser <name> -Properties IsCriticalSystemObject,proxyAddresses,userPrincipalName,distinguishedName` returns the object.

Implement if missing: Install RSAT / the ActiveDirectory module (`Add-WindowsCapability -Online -Name Rsat.ActiveDirectory.DS-LDS.Tools~~~~0.0.1.0` on a client, or the AD DS role tools on a server) and use a read account. No write is needed.

Checklist

- Access to `Cloud Sync – Provisioning Agent Host` (R1) — can read services + Trace folder
- Local administrator on the host (R2) — run scripts elevated
- AADCloudSyncTools module present on the host (R3) — for the log export

- Microsoft Graph read scopes (`AuditLog.Read.All`, `Directory.Read.All`, `Synchronization.Read.All`) (R4)
- Entra read role (Hybrid Identity Administrator or Global Reader) (R5)
- The failing user's `ObjectGuid`/UPN and the intended in-scope OU DNs recorded (R6)
- Active Directory read access + the ActiveDirectory module (R7)

Global Requirements — Entra ID Solutions (category-level)

These requirements apply to every solution in the Entra ID Solutions category. They are published once at the category level so individual listings stay small. Confirm these before deploying any identity solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *The identity plane is high risk. A mistake here breaks sign-in for everyone, so every change is staged and a break-glass account is kept ready before you touch anything.*

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID tenant. Why: It is the cloud identity backbone every other identity service builds on. Verify: Sign in to `https://entra.microsoft.com` as an administrator. Implement: Create a Microsoft 365 or Azure subscription, which provisions a tenant.

G2. Entra ID licensing (P1, P2 where noted)

Description: Microsoft Entra ID P1 (or P2 for risk-based and PIM features). Why: Hybrid features such as seamless sign-in, password writeback, and Conditional Access require P1+. Verify: M365 admin center -> `Billing -> Licenses`; confirm Entra ID P1/P2 (often bundled in M365 E3/E5 or EMS). Implement: Assign M365 E3/E5, EMS E3/E5, or standalone Entra ID P1/P2.

G3. Administrative roles

Description: Hybrid Identity Administrator and Global Administrator for the operator. Why: Configuring directory synchronization and authentication methods needs these elevated roles. Verify: Entra -> `Roles and administrators`; confirm `Hybrid Identity Administrator` (and a Global Administrator for the one-time first-sync consent). Implement: Assign `Hybrid Identity Administrator`; reserve `Global Administrator` for tenant-wide and first-sync changes.

G4. Break-glass (emergency access) accounts

Description: At least two cloud-only emergency access accounts, excluded from Conditional Access and MFA-broad policies. Why: If sync, federation, or an auth-method change locks out admins, break-glass accounts get you back in. Verify: Entra -> `Users`; confirm two cloud-only accounts (`@onmicrosoft.com`) with strong stored credentials and documented use. Implement: Create two cloud-only Global Administrator accounts; exclude them from policies; store credentials securely and monitor sign-ins.

G5. Healthy on-premises Active Directory

Description: A functional AD forest/domain with healthy replication and DNS. Why: Synchronization reads from AD; replication or DNS faults corrupt or stall the sync. Verify: Run `dcdiag` and `repadmin /replsummary` on a domain controller with no errors; run `HYBID-Discovery.ps1`. Implement: Resolve AD replication, DNS, and time-sync issues before synchronizing.

G6. An Entra Connect host server

Description: A domain-joined Windows Server (2016+), patched, for Microsoft Entra Connect Sync (or a lightweight host with the Cloud Sync agent). Why: Connect Sync runs as a service on this host and must reach both AD and Entra ID. Verify: A supported, domain-joined server with TLS 1.2, .NET, and PowerShell available; run `HYBID-Discovery.ps1`. Implement: Provision a dedicated server (not a domain controller for production), patch it, and enable TLS 1.2.

G7. UPN / domain hygiene

Description: User principal names use a public, verified, routable domain (not `.local`). Why: A non-routable UPN suffix produces sign-in problems and `onmicrosoft.com` fallbacks in the cloud. Verify: Entra -> `Identity -> Settings -> Domain names` shows the public domain Verified; AD UPN suffixes match it; run `HYBID-Discovery.ps1`. Implement: Add and verify the public domain in Entra; add the matching UPN suffix in AD and update user UPNs.

G8. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS (443) from the Connect host to Entra ID, plus required Microsoft 365 service URLs. Why: Synchronization, authentication-agent traffic, and tokens depend on these endpoints. Verify: From the Connect host, reach `https://login.microsoftonline.com` and the Microsoft 365 published URL/IP ranges. Implement: Allow the Microsoft 365 / Entra service URLs and IP ranges through the proxy/firewall.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

Diagnostic Workflow

This is the core deliverable: the ordered workflow for a **cloud sync "users skipped / JoinNotFound / not provisioned"** incident. It adapts the category-standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model to a diagnostic shape:

Collect → **Analyze** → **Remediate** → **Verify.**

During an incident, start here. Work it in order — **Collect once** on the agent host, then **Analyze** the area the symptom points to with the focused tools, **Remediate** in the owning plane (portal / AADCloudSyncTools / AD), and **Verify** by re-running provision on demand and reading the provisioning log until the object provisions with **action = Create**.

> *Names used verbatim (from `01-solution-overview.md` / `02-requirements.md`): `Cloud Sync – AD to Entra – Configuration`, `Cloud Sync – In-Scope Users – OU`, `Cloud Sync – Pilot Scope – Pilot`, `Cloud Sync – Provisioning Agent Host`, and Microsoft's **"Microsoft Entra Provisioning Agent"**, **"Microsoft Entra Connect Agent Updater"**, and **"Microsoft***

Entra AD Synchronization Service".

Phase 1 — Collect (always start here)

Run the collector **once** on `Cloud Sync – Provisioning Agent Host` to snapshot the agent state and gather the logs into one bundle — so you spend the incident reasoning, not gathering.

```
powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Collect-Diagnostics.ps1 -OutFolder .\CloudSyncDiag
```

Run it **elevated** (R2) so it can read the agent services, the Trace folder, and export the logs. The collector captures:

- the **"Microsoft Entra Provisioning Agent"** and **"Microsoft Entra Connect Agent Updater"** service states (should be **Running**);
- the PowerShell **execution policy** (an over-restrictive or `Unrestricted` policy is noteworthy);
- the **proxy** configuration presence (WinHTTP / agent config), which affects outbound reachability;
- the agent **Trace** folder (`C:\ProgramData\Microsoft\Azure AD Connect Provisioning Agent\Trace`);
- and, **if the AADCloudSyncTools module is present**, a full `Export-AADCloudSyncToolsLogs` bundle.

Everything is zipped to a timestamped `Cloud Sync – Diagnostics – <timestamp>` folder. The collector makes **no changes**.

Gate (Collect): the bundle wrote successfully and the two agent services are named with their state. If the host can't run PowerShell elevated, resolve R1/R2 first.

Phase 2 — Analyze (run the focused check for the flagged area)

Pick the branch by **symptom**. Each focused tool prints a **PASS/FAIL** verdict (or the authoritative provisioning-log reason) and the exact remedy.

Symptom → tool map

Symptom	Where the truth is	Run this tool	What it isolates
Provision on demand shows **Evaluate → "User was skipped", JoinNotFound**	Provisioning logs	`CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid <guid>`	The **authoritative** per-object action / status / skipReason + step detail

Symptom	Where the truth is	Run this tool	What it isolates
Nothing is being created fleet-wide; job may be stuck	Sync job / SP / org	`CLDSYNCDX-Check-Provisioning.ps1`	Job **Active/Enabled/Quarantined**, SP present, `onPremisesSyncEnabled`, deletion-threshold
The **agent shows unhealthy** / recently stopped syncing	Agent host	`CLDSYNCDX-Check-Agent.ps1`	Services Running · execution policy · outbound 443 · proxy/config
A **specific user** never provisions (skipped / not created)	Active Directory	`CLDSYNCDX-Test-Object.ps1 -Identity <user> -InScopeOU <dn[]>`	Critical-object · out-of-scope OU · non-routable UPN · possible duplicate
You **just changed scope/config** and still nothing	Sync job + AD object	`CLDSYNCDX-Check-Provisioning.ps1` **then** `CLDSYNCDX-Test-Object.ps1`	Whether a **restart** is needed to clear watermarks, and whether the OU is now in scope

Branch A — read the authoritative reason (start here for JoinNotFound)

```
powershell -File .\CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid 11111111-2222-3333-4444-555555555555
```

or by UPN:

```
powershell -File .\CLDSYNCDX-Get-ProvisioningLog.ps1 -UserPrincipalName user@contoso.com
```

It queries the **provisioning logs** (Microsoft Graph `auditLogs/provisioning`) for that object and prints the latest entries with the **provisioningAction** (create/update/delete/other), the **status** (success / skipped / failure), the **skipReason** (e.g. `JoinNotFound`), and the per-step detail (Import / Scope / Match / Evaluate). This is the authoritative statement of *what the service decided and why* — everything else in Analyze is about explaining that reason.

Branch B — job / service-principal / org health

```
powershell -File .\CLDSYNCDX-Check-Provisioning.ps1
```

It finds the cloud sync **synchronization job**, reports whether it is **Active / Enabled / Quarantined** (with the quarantine reason and escrow count if any), whether the **"Microsoft Entra AD Synchronization Service"** service principal exists, and the org's `onPremisesSyncEnabled`. It flags `AzureDirectoryServiceAuthorizationFailed` (missing SP) and object-deletion-threshold symptoms. Read-only — it never restarts the job.

Branch C — agent health on the host

```
powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Check-Agent.ps1
```

It checks, with PASS/FAIL and a remedy per FAIL: the two agent **services are Running**, the **execution policy** is workable (not `Unrestricted`, ideally `RemoteSigned`/`Undefined`), a **sample outbound 443** endpoint is reachable, and the **agent config / proxy** file is present. The first FAIL is usually why the agent went unhealthy.

Branch D — is this specific object even a candidate?

```
powershell -File .\CLDSYNCDX-Test-Object.ps1 -Identity jsmith `
    -InScopeOU 'OU=Cloud Sync – In-Scope Users – OU,DC=contoso,DC=com','OU=Cloud Sync – Pilot Scope – Pilo
```

It reports, for the user: `IsCriticalSystemObject` (excluded by design if true), whether its OU is **under one of the supplied in-scope OU DNs** (out-of-scope if not), whether its **UPN suffix** is a routable/verified domain, and an **advisory** on a possible same-UPN/`proxyAddresses` object that would already exist in the tenant. Read-only against AD.

Gate (Analyze): you can name the condition — either the provisioning log states the reason (`JoinNotFound` + the failing step), or a focused tool reports a specific FAIL with a named remedy. Do not guess past it; fix it and re-check.

Phase 3 — Remediate (in the owning plane)

The toolkit is **read-only** — it never changes the object, tenant, configuration, or agent. Apply the indicated fix in the plane that owns it. Typical remedies by finding:

Finding	Where to fix	Typical remedy
Target already exists (prior sync method / cloud-only same UPN)	Entra ID	Reconcile the duplicate: remove/rename the stale cloud object, or let the soft-match join it; align `userPrincipalName` / `proxyAddresses` so exactly one target matches
Configuration not **Enabled**	Portal (Cloud sync)	Enable `Cloud Sync – AD to Entra – Configuration`
User's OU **not in scoping filters**	Portal (Cloud sync)	Add the user's OU to the configuration's scoping filters (or move the user into `Cloud Sync – In-Scope Users – OU`)
Scope/config changed but objects not re-discovered	Portal / AADCloudSyncTools	**Restart provisioning** on the configuration to clear watermarks so objects are re-evaluated
Object is `IsCriticalSystemObject` / replication victim	(by design)	Test with a normal user; built-in accounts are filtered out intentionally
Job **quarantined** (e.g. bad admin creds)	Portal / AADCloudSyncTools	**Clear quarantine** / **Restart** the job after fixing the underlying auth failure; `Repair-AADCloudSyncToolsAccount` for the agent account

Finding	Where to fix	Typical remedy
Object-deletion threshold blocking (Connect + cloud sync coexistence)	AADCloudSyncTools	`Disable-AADCloudSyncToolsDirSyncAccidentalDeletionPrevention -tenantId <tid>` (then re-enable the safeguard after the run)
`AzureDirectoryServiceAuthorizationFailed` — SP missing	Microsoft Graph	Re-enable sync to recreate the **"Microsoft Entra AD Synchronization Service"** SP: `PATCH /organization/<tid> { "onPremisesSyncEnabled": true }`
Agent service stopped / unhealthy / no outbound 443	Agent host / network	Start the **"Microsoft Entra Provisioning Agent"** + **"Microsoft Entra Connect Agent Updater"** services; open outbound **80/443** to the documented endpoints; restart the agent

Several of these touch tenant-wide or outward-facing state (the org sync flag, quarantine, the deletion safeguard). Remediate under **change control** — see `07-best-practices.md`. Exact cmdlet syntax and portal navigation for the cloud sync service are **deferred to Microsoft's official documentation**.

Phase 4 — Verify (re-run until the object provisions)

After a fix, re-run the focused tool for the branch you worked, then run the decisive test: **provision on demand** the user again and read the provisioning log.

```
powershell -File .\CLDSYNCDX-Check-Provisioning.ps1                # job Active/Enabled, not quarantined
powershell -File .\CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid <guid>   # action now = Create / Update,
```

Then prove it **in the tenant**: run **provision on demand** for the user in **Entra Connect** → **Cloud sync** → the configuration, confirm the **Evaluate action** now shows **Create** (or **Update**), and confirm the user **appears in Entra ID** with the correct UPN. For the scope fix, let a **scheduled cycle** run (or restart provisioning) and confirm the previously out-of-scope user is created. Confirm each gate in `05-testing.md`.

Gate (Verify): the provisioning log for the object shows **action = Create/Update** (no longer **skipped / JoinNotFound**), the job is **Active/Enabled and not quarantined**, and the user is present in Entra ID. Only then is the incident closed. Record the root-cause condition on the ticket (see `07-best-practices.md`, closing the loop).

Testing — Confirming the Toolkit and the Fix

Two things get tested here. First, that the **toolkit itself runs correctly** — it must read a **known-good** object as provisioning and a **known-bad** object (the JoinNotFound user) as skipped, so you can trust its verdicts. Second, that a **fix actually cleared** the failing condition — the decisive test is that, after remediation, **provision on demand creates the user** and the **provisioning log shows action = Create**. Each check has an explicit **pass/fail gate**. Everything here is **read-only** — you observe; you never force a state.

> *Reuse names verbatim:* `Cloud Sync – AD to Entra – Configuration`, `Cloud Sync – In-Scope Users – OU`, `Cloud Sync – Pilot Scope – Pilot`, `Cloud Sync –`

`Provisioning Agent Host`, *and Microsoft's* ***"Microsoft Entra Provisioning Agent"***, ***"Microsoft Entra Connect Agent Updater"***, ***"Microsoft Entra AD Synchronization Service"***.

Test matrix overview

#	Test	Where	Pass gate
T1	Collector runs and writes a bundle	Agent host	Bundle written; both agent services named with state
T2	Agent-health check is accurate	Agent host	Services Running / policy / 443 / proxy verdicts match reality
T3	Provisioning-log read returns the reason	Operator workstation	The object's latest action/status/skipReason printed (JoinNotFound seen on the bad object)
T4	Job / SP / org check is accurate	Operator workstation	Job Active/quarantine, SP present, `onPremisesSyncEnabled` verdicts match the portal
T5	Object test classifies correctly	Domain member	Critical / in-scope / UPN-routable / duplicate-advisory match the AD facts
T6	Known-good object reads as provisioning	Reference user	Provision on demand shows Create/Update; log agrees
T7	Known-bad object reads as skipped	JoinNotFound user	Provision on demand shows "User was skipped / JoinNotFound"; the tools name the cause
T8	Post-fix re-run provisions the user	Fixed user	Provision on demand = **Create**; log shows action=Create; user in Entra ID

T1 — Collector runs and writes a bundle

Steps: On `Cloud Sync – Provisioning Agent Host`, run `CLDSYNCDX-Collect-Diagnostics.ps1 -OutFolder .\CloudSyncDiag` elevated. **PASS:** The console shows the two agent service states and a timestamped `Cloud Sync – Diagnostics – <timestamp>` zip/folder is written (with the Trace folder, and the AADCloudSyncTools export if the module is present). **FAIL:** The script errors or can't read the Trace folder / services → run elevated (R2) and re-test.

T2 — Agent-health check is accurate

Steps: Run `CLDSYNCDX-Check-Agent.ps1` on the host and compare to `services.msc` and a manual reachability test. **PASS:** The Running/stopped, execution-policy, outbound-443, and proxy verdicts match what the host actually shows. **FAIL:** A verdict disagrees with reality (e.g. reports a service stopped that is running) → confirm elevation and re-test.

T3 — Provisioning-log read returns the authoritative reason

Steps: Run `CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid <bad-object-guid>` for the user that is skipped. **PASS:** The tool prints the latest provisioning entry with its **action**, **status = skipped**, and **skipReason = JoinNotFound** plus the Import/Scope/Match/Evaluate step detail — matching what **Entra Connect** → **Cloud sync** → **Logs** shows for that object. **FAIL:** No entries returned for an object you know was evaluated → confirm the `ObjectGuid` is correct and the Graph scopes (`AuditLog.Read.All`) are consented; try `-UserPrincipalName`.

T4 — Job / SP / org check is accurate

Steps: Run `CLDSYNCDX-Check-Provisioning.ps1` and compare to **Entra Connect** → **Cloud sync** → the configuration's status and agent health. **PASS:** The job **Active/Enabled/Quarantined** verdict, the **"Microsoft Entra AD Synchronization Service"** SP present/absent verdict, and `onPremisesSyncEnabled` match the portal. **FAIL:** A verdict disagrees with the portal → confirm the `Synchronization.Read.All` / `Directory.Read.All` scopes and that you targeted the correct configuration/SP.

T5 — Object test classifies correctly

Steps: Run `CLDSYNCDX-Test-Object.ps1 -Identity <user> -InScopeOU <in-scope-DNs>` against a known in-scope normal user and, separately, a built-in account. **PASS:** The normal user reports **not critical**, **in scope**, **UPN routable**; the built-in account reports `IsCriticalSystemObject = true` **(excluded by design)**. The duplicate advisory matches whether a same-UPN object exists. **FAIL:** A classification disagrees with the AD facts (`Get-ADUser … -Properties IsCriticalSystemObject,proxyAddresses,distinguishedName`) → reconcile the supplied in-scope OU DNs.

T6 — Known-good object reads as provisioning (no false positives)

Steps: For a user that provisions correctly, run **provision on demand** in the portal and `CLDSYNCDX-Get-ProvisioningLog.ps1` for the same object. **PASS:** Provision on demand shows **Create** (first time) or **Update** (already joined), and the log agrees — confirming the tools read a working object as working. **FAIL:** The tools report skipped on a user that is actually in Entra ID → confirm you targeted the right object and that the tenant object is the one cloud sync owns (`onPremisesSyncEnabled`).

T7 — Known-bad object reads as skipped (no false negatives)

Steps: For the JoinNotFound user, run **provision on demand** and the toolkit (Get-ProvisioningLog + the relevant Analyze branch). **PASS:** Provision on demand shows **Import ✓ / In scope ✓ / Match ✓** →

Evaluate: "User was skipped", JoinNotFound, and the toolkit names the underlying cause (already-exists / out-of-scope for the scheduled cycle / critical object / quarantine). **FAIL:** The toolkit reports all clean on a user that clearly isn't provisioning → the cause is beyond the checks covered (escalate with the collected bundle and the provisioning-log output).

T8 — Post-fix re-run provisions the user (the decisive test)

Steps: After remediating (Phase 3), run **provision on demand** for the user again, then `CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid <guid>` and `CLDSYNCDX-Check-Provisioning.ps1`. **PASS:** Provision on demand's **Evaluate action = Create** (or **Update**), the provisioning log shows **action = Create/Update** (no longer skipped/JoinNotFound), the job is **Active/Enabled and not quarantined**, and the **user now exists in Entra ID** with the correct UPN. For a scope fix, a scheduled cycle creates the previously out-of-scope user. **FAIL:** Still skipped → return to the matching branch in `03-implementation.md` / `06-troubleshooting.md`.

Overall exit criteria

The toolkit passes T1–T7 (it reads a known-good object as provisioning and the known-bad object's true skip reason), and for a closed incident **T8** confirms provision on demand now creates the user, the provisioning log shows **action = Create**, and the user is present in Entra ID. Record results against each gate before closing.

Troubleshooting — Cloud Sync "Users Skipped / JoinNotFound / Not Provisioned" Symptom → Cause → Fix Matrix

This is the heart of the product: a comprehensive **symptom** → **likely cause** → **fix** matrix for **Microsoft Entra Connect cloud sync** when users are **skipped, not created, or reported JoinNotFound**. Work top-down within a section; the first cause listed is the most common. Everything here stays **read-only** on the diagnostic side — the *fix* changes state and is applied in the owning plane (see `03-implementation.md` Phase 3).

> ***Product boundary:*** *cloud sync's* ***provisioning-service internals*** *— the exact pipeline behaviour, the full set of provisioning-log reason codes, and the internals of the* ***AADCloudSyncTools*** *cmdlets — are* ***deferred to Microsoft's official documentation****. This guide resolves the* ***agent-host, Active Directory, Entra, and Microsoft Graph*** *side and tells you which condition a symptom points at.*
>
> *Reuse names verbatim:* `Cloud Sync – AD to Entra – Configuration`, `Cloud Sync – In-Scope Users – OU`, `Cloud Sync – Pilot Scope – Pilot`, `Cloud Sync – Provisioning Agent Host`, *and Microsoft's* ***"Microsoft Entra Provisioning Agent"****,* ***"Microsoft Entra Connect Agent Updater"****,* ***"Microsoft Entra AD Synchronization Service"****.*

§The core symptom — provision on demand shows "User was skipped, JoinNotFound"

Symptom: In **Entra Connect** → **Cloud sync** → the configuration → **Provision on demand**, the steps show **Import ✓ / Determine if in scope ✓ / Match user ✓**, then **Determine object action (Evaluate)** → **"User was skipped"** with **SkipReason: JoinNotFound** — *"not a newly discovered entry to be provisioned … nor one with an update to flow to a previously matched target."* `CLDSYNCDX-Get-ProvisioningLog.ps1` prints the same reason for the object's `ObjectGuid`.

What it means: the evaluate step found **no existing target to join** *and* the object is **not a valid new-provision candidate at that moment** — so there is nothing to create and nothing to update. JoinNotFound is the *result*; the sections below are the *causes*. Work them in order.

1. **A matching target already exists in Entra ID (most common).** If an object with the same

`userPrincipalName` / `proxyAddresses` / anchor already exists — commonly a **leftover from a prior sync method** (an earlier Entra Connect *Sync* deployment) or a **cloud-only** account — the evaluate step resolves to a soft-match/no-op and creates nothing. Run `CLDSYNCDX-Test-Object.ps1` for the duplicate advisory and check Entra ID for an existing object with that UPN/proxyAddress. **Fix:** reconcile the duplicate — remove or rename the stale cloud object, or let the soft-match join it; align `userPrincipalName` / `proxyAddresses` so exactly one target matches. (`Match user ✓` on the on-demand screen does **not** mean "matched to a new create"; it means the match step ran.)

1. **The object isn't a valid create candidate for the scheduled cycle (scope).** Provision on demand

ignores scope, so it can evaluate a user the scheduled cycle would never create. If the user's OU is **not in the scoping filters** of `Cloud Sync – AD to Entra – Configuration`, the scheduled cycle skips it — even though on-demand ran. Run `CLDSYNCDX-Test-Object.ps1 -InScopeOU <dns>`. **Fix:** add the user's OU to the configuration's scoping filters (or move the user into `Cloud Sync – In-Scope Users – OU`), then **restart provisioning** to clear watermarks.

1. **The object is excluded by design.** `IsCriticalSystemObject = true` (built-in accounts/groups) and

replication-victim objects are filtered out. Testing with one of those *always* looks like JoinNotFound. `CLDSYNCDX-Test-Object.ps1` reports the critical flag. **Fix:** test with an ordinary user in a known in-scope OU; the exclusion is intentional.

1. **A stale watermark after a recent change.** If you just changed scope, mapping, or the

configuration, already-seen objects aren't re-evaluated automatically. **Fix: restart provisioning** on the configuration so objects are re-discovered.

§Nothing is being created at all (fleet-wide, not one user)

Symptom: No users are provisioning; it isn't limited to one object. `CLDSYNCDX-Check-Provisioning.ps1` shows the job or org state.

1. **■ Stale watermark after an earlier quarantine or after a fix — the #1 "healthy but creates

nothing" cause. **If the job** was quarantined earlier **(or the tenant domain finished verifying *after* the first cycle, or scope/mapping changed), cloud sync already** saw and parked **those objects and advanced its** watermark **past them. Now that the job is** Healthy**, those users are no longer** "newly discovered" **— so** every **account re-evaluates to** JoinNotFound **and nothing is created.** Clearing the quarantine does NOT fix this **— it doesn't re-open already-seen objects (and if it's not currently quarantined, *Clear quarantine* is greyed out entirely).** Fix: **on the configuration/agent page, click**

Restart sync **— a *full* restart that resets** watermarks + escrows + quarantine**, so every in-scope object is re-discovered as new. Graph equivalent:** `POST /servicePrincipals/{spId}/synchronization/jobs/{jobId}/restart` **resetting** `Watermarks,Escrows,Quarantine`**. Then re-run** Provision on demand **on one user** → **expect** action = Create**. (This is the classic pattern when someone reports "it was quarantined earlier, now it shows OK, but the same JoinNotFound appears for all users.")

1. **The configuration is not Enabled.** A configuration that was created but left disabled never runs

the scheduled cycle. **Fix:** enable `Cloud Sync – AD to Entra – Configuration`.

1. **The synchronization job is quarantined.** If calls to the target consistently fail (commonly bad

admin credentials for the agent, or a persistent tenant-side error), the job is **quarantined** and nothing exports. `CLDSYNCDX-Check-Provisioning.ps1` reports the quarantine and its reason/escrow. **Fix:** resolve the underlying auth failure (e.g. `Repair-AADCloudSyncToolsAccount`), then **Restart sync** (not just *Clear quarantine* — see cause 1). Clearing quarantine without fixing the underlying error just re-quarantines.

1. **New-tenant domain move — verify the domain first.** If you moved a custom domain from an old tenant

into this new tenant (removing it from the old one), confirm it is **Verified** in the new tenant (**Entra ID** → **Settings** → **Domain names**) *before* expecting creates — users' UPN suffix must be a verified domain at evaluation time. If it verified only after the first (parked) cycle, do the **Restart sync** in cause 1 so the now-valid users are re-discovered.

1. **An object-deletion threshold is blocking the run.** In **Connect + cloud sync coexistence** (both

exporting to one tenant), Entra's accidental-deletion-prevention threshold can block the export. **Fix:** investigate why the run wants to change that many objects, then `Disable-AADCloudSyncToolsDirSyncAccidentalDeletionPrevention -tenantId <tid>` for the run and **re-enable** the safeguard afterward.

1. `AzureDirectoryServiceAuthorizationFailed` **— the sync service principal is missing.** If the

"Microsoft Entra AD Synchronization Service" service principal is absent, the agent can't authorize and the whole job fails. `CLDSYNCDX-Check-Provisioning.ps1` flags the missing SP and reads `onPremisesSyncEnabled`. **Fix:** re-enable directory sync to recreate the SP — `PATCH /organization/<tid> { "onPremisesSyncEnabled": true }`.

1. **The agent is unhealthy or offline.** If the agent isn't running or can't reach the service, no

cycle runs — see §Agent health.

§Agent health — the provisioning agent isn't working

Symptom: The agent shows **not healthy** (not green) in **Entra Connect** → **Cloud sync** → **agents**, or sync recently stopped. `CLDSYNCDX-Check-Agent.ps1` shows the first FAIL on the host.

1. **A required service is stopped.** The **"Microsoft Entra Provisioning Agent"** and **"Microsoft

Entra Connect Agent Updater" **services must both be** Running**.** Fix:** start the stopped service (and set it to Automatic); if it won't stay running, collect the Trace folder with `CLDSYNCDX-Collect-Diagnostics.ps1` and review.

1. **Outbound 80/443 blocked.** The agent needs outbound **80/443** to the documented Microsoft

endpoints. `CLDSYNCDX-Check-Agent.ps1` tests a sample 443 endpoint. **Fix:** open the documented URLs through the proxy/firewall from `Cloud Sync – Provisioning Agent Host`.

1. **Proxy misconfiguration.** If the host uses a proxy, the agent must be configured for it. The check

reports whether a proxy/agent config is present. **Fix:** configure the agent's proxy per Microsoft guidance and restart the agent services.

1. **Execution policy blocks the tooling.** An over-restrictive (or `Unrestricted`, which is a

different risk) execution policy can interfere with running the diagnostic/AADCloudSyncTools scripts. The check reports the policy. **Fix:** run the tools with `-ExecutionPolicy Bypass` for the session, or set a workable per-process policy; don't leave the machine `Unrestricted`.

1. **Agent version out of date.** A very old agent can misbehave; the **Agent Updater** service keeps it

current. **Fix:** confirm the updater service is running, or reinstall the current agent — **deferred to Microsoft documentation** for the install steps.

§Scope & matching — the object is not a create candidate

Symptom: One user (or one OU's users) never appears in Entra ID, though the agent is healthy and other users provision.

1. **OU not in the scoping filters.** The single most common scope cause: the user's OU isn't included

in the configuration's scoping filters, so the **scheduled cycle** never creates them (even though provision on demand can). Run `CLDSYNCDX-Test-Object.ps1 -InScopeOU <dns>`. **Fix:** add the OU to the scope (or move the user into `Cloud Sync – In-Scope Users – OU`), then **restart provisioning**.

1. **Watermark not cleared after a scope change.** You added the OU but didn't restart — objects already

seen aren't re-evaluated. **Fix:** restart provisioning.

1. **Non-routable / unverified UPN suffix.** A user whose UPN suffix is a non-routable domain (e.g.

`.local`) or a domain not **Verified** in the tenant can fail to provision cleanly or land on the `onmicrosoft.com` fallback. `CLDSYNCDX-Test-Object.ps1` flags a non-routable/unverified suffix. **Fix:** add and verify the public domain in Entra, add the matching UPN suffix in AD, and update the user's UPN.

1. **Duplicate attribute conflict.** Two AD objects (or an AD object and a cloud object) sharing a

`userPrincipalName` or `proxyAddresses` value cause match ambiguity and a skip. **Fix:** resolve the duplicate so exactly one object holds the value.

1. **Group/contact vs user expectations.** Cloud sync provisions the object types the configuration is

set for; confirm you're testing the right object type for the configuration's mapping.

§Already-exists — the "prior sync method" trap

Symptom: The user clearly exists in AD and is in scope, but cloud sync creates nothing; the object already has a counterpart in Entra ID.

1. **Leftover from an earlier Entra Connect *Sync* deployment.** If this tenant was previously synced

with the older Connect Sync client, cloud-side objects (possibly still marked on-prem-synced) already exist. Cloud sync matches to them and does nothing new — exactly the scenario behind an engineer who "last set this up years ago with a different sync method." **Fix:** decide the authority: retire the old objects/soft-match cleanly, or align anchors so cloud sync owns them; confirm only one sync engine is authoritative for a given object.

1. **Cloud-only account with the same UPN/proxyAddress.** A manually-created cloud user collides with

the AD user on `userPrincipalName` / `proxyAddresses`. **Fix:** reconcile (convert/soft-match or remove the cloud-only object), then re-run provision on demand.

1. **Object was deleted but is in the recycle bin.** A soft-deleted Entra object retaining the UPN can

block a fresh create. **Fix:** permanently remove or restore the soft-deleted object as appropriate, then re-provision.

§Coexistence & thresholds — Connect Sync and cloud sync together

Symptom: Both Connect Sync and cloud sync export to the same tenant and changes are blocked or inconsistent.

1. **Accidental-deletion threshold tripped.** A large delta (from a scope change or a mis-filter) trips

the export deletion safeguard and blocks the run. **Fix:** confirm the change is intended, then temporarily `Disable-AADCloudSyncToolsDirSyncAccidentalDeletionPrevention -tenantId <tid>`, let the run complete, and **re-enable** the safeguard.

1. **Two engines contending for the same objects.** If Connect Sync and cloud sync both target

overlapping objects, they contend. **Fix:** scope each engine to a disjoint set of objects/OUs so only one owns each object; coexistence is supported but must be scoped deliberately.

§Reading the authoritative reason — provisioning logs

The **provisioning logs** are the source of truth. Read them by the AD `ObjectGuid` — the reliable key — rather than by display name.

1. **Use the ObjectGuid.** `CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid <guid>` returns the object's

entries with **action**, **status**, **skipReason**, and step detail. In the portal: **Entra Connect** → **Cloud sync** → **Logs**, filter by the object.

1. **No log entry for the object.** If nothing appears, the object was never evaluated — it's likely

out of scope for the scheduled cycle (run provision on demand to force an evaluation and generate a log entry), or the job isn't running (see §Nothing is being created).

1. **The reason isn't JoinNotFound.** The tool prints whatever the real reason is (a different skip

reason, a failure with an error code such as `AzureDirectoryServiceAuthorizationFailed`, or a success). Match the printed reason to the section above.

§Script / tooling issues

1. `Connect-MgGraph` **fails or lacks scopes.** Install the SDK (`Install-Module Microsoft.Graph -Scope

CurrentUser) and consent to the read scopes the script header lists (AuditLog.Read.All, Directory.Read.All, Synchronization.Read.All`).

1. **AADCloudSyncTools not found on the host.** `CLDSYNCDX-Collect-Diagnostics.ps1` still gathers the

services state and the Trace folder without it; install the module (`Install-Module AADCloudSyncTools`) and `Connect-AADCloudSyncTools` for the full log export.

1. **Access denied reading the Trace folder or services.** Run the host-side scripts **elevated** (R2);

they're read-only, but elevation is needed for visibility.

1. **The ActiveDirectory module is missing for the object test.** Install RSAT / the ActiveDirectory

module and run `CLDSYNCDX-Test-Object.ps1` on the agent host or a domain member.

1. **The synchronization job or SP can't be found.** Cloud sync's job lives on the provisioning service

principal for the configuration; `CLDSYNCDX-Check-Provisioning.ps1` searches for it. If your tenant names or structures it differently, pass the service principal id explicitly and confirm the configuration exists under **Entra Connect** → **Cloud sync**.

When to stop diagnosing and escalate to Microsoft

Diagnosis here is read-only by design. When the **agent host, Active Directory, scope, matching, and job** side is confirmed clean (the checks pass, the object is in scope, no duplicate, the job is Active and not quarantined) but provision on demand still reports **JoinNotFound** or a create still doesn't happen, treat it as a **cloud sync service issue** and open a Microsoft support case. Hand over: the `CLDSYNCDX-Collect-Diagnostics.ps1` bundle (agent services + Trace + AADCloudSyncTools export), the `CLDSYNCDX-Get-ProvisioningLog.ps1` output for the object's `ObjectGuid`, the `CLDSYNCDX-Check-Provisioning.ps1` job/SP/org state, and a statement that the on-prem/scope/match side is clean. A complete package turns a multi-round case into a single decisive one.

7. Best Practices — Cloud Sync Provisioning Triage

- **Read the provisioning log first.** The **provisioning logs** (searchable by AD `ObjectGuid`) are

the authoritative statement of what the service decided and why. Start every "not provisioning" case by reading the object's log entry — don't theorize before you've read `JoinNotFound` (or whatever the real reason is) with your own eyes.

- **Confirm which engine you actually run.** Cloud sync and Connect Sync are different products with

different tooling. Before touching anything, confirm this is **cloud sync** (a provisioning agent, a configuration under **Entra Connect** → **Cloud sync**) and not **Connect Sync** (the `ADSync` service) — applying Connect Sync habits to a cloud sync configuration wastes the incident.

- **Remember provision on demand ignores scope.** On-demand runs the pipeline for one object regardless

of the scoping filters, so an on-demand skip is usually *not* about scope for that run — while the **scheduled cycle only creates in-scope users**. Diagnose scope against the scheduled behaviour.

- **Suspect a pre-existing target when history includes another sync method.** A leftover from an

earlier Connect Sync deployment, or a cloud-only account with the same `userPrincipalName` / `proxyAddresses`, is the most common reason an object matches to nothing new and is skipped. Reconcile duplicates before assuming the engine is broken.

- **After any scope or configuration change, restart provisioning.** Cloud sync uses watermarks to skip

already-seen objects; changing scope does not automatically re-evaluate them. Restart provisioning to clear watermarks so objects are re-discovered.

- **Never test with a built-in account.** `IsCriticalSystemObject` objects (built-in users and groups)

and replication-victim objects are filtered out by design. Always test provisioning with an ordinary user in a known in-scope OU.

- **Treat quarantine as a symptom, not a cause.** A quarantined job means the target consistently

rejected calls (often bad admin credentials). Fix the underlying auth failure, then clear quarantine / restart — clearing quarantine without fixing the cause just re-quarantines.

- **Respect the object-deletion threshold.** In Connect + cloud sync coexistence the accidental-

deletion safeguard can block a run. Investigate *why* the run wants to change that many objects before disabling the safeguard, and **re-enable it** after the run.

- **Keep the agent healthy and reachable.** The **"Microsoft Entra Provisioning Agent"** and

"Microsoft Entra Connect Agent Updater" services must be Running, the agent Healthy in the portal, and outbound **80/443** to the documented endpoints open. An unhealthy agent silently stops exporting.

- **Diagnose read-only; remediate under change control.** The tenant-wide actions (re-enabling org

sync, clearing quarantine, disabling the deletion safeguard) affect every synced object. Keep diagnosis read-only, and make the change through your change process with a named approver.

- **Close the loop.** Record the root-cause condition on the ticket (already-exists / out-of-scope /

critical object / quarantine / SP missing / threshold / agent), the `ObjectGuid`, and the provisioning-log action before and after. A named root cause makes the next case faster.

8. Deliverables Summary — Troubleshooting Microsoft Entra Connect Cloud Sync (JoinNotFound)

Documentation

Document	Purpose
Executive Summary	The problem, what is in the box, and the outcome
1. Solution Overview	Cloud sync vs. Connect Sync, what JoinNotFound means, the pipeline, the conditions, the five tools
2. Requirements Guide	Every read-access prerequisite with Verify / Implement-if-missing
Global Requirements (Entra ID Solutions)	Category-wide prerequisites, published once
3. Diagnostic Workflow	The ordered Collect → Analyze → Remediate → Verify process, symptom → tool map, and gates
5. Testing Guide	Validation procedures with pass/fail gates (decisive test: provision on demand = Create)
6. Troubleshooting Guide	The comprehensive symptom → cause → fix matrix — the heart of the product
7. Best Practices	Field-proven triage habits and clean escalation

PowerShell diagnostic tools (all read-only)

Script	What it does
`CLDSYNCDX-Collect-Diagnostics.ps1`	On the agent host: agent services, execution policy, proxy, Trace folder, and (if present) `Export-AADCloudSyncToolsLogs`, zipped to a timestamped folder
`CLDSYNCDX-Check-Agent.ps1`	On the agent host: PASS/FAIL agent-health checks (services Running, execution policy, outbound 443, proxy/config) with a remedy per FAIL
`CLDSYNCDX-Get-ProvisioningLog.ps1`	Microsoft Graph: the provisioning-log entries for an `ObjectGuid`/`UPN` — action, status, and **skipReason** (e.g. JoinNotFound) with step detail
`CLDSYNCDX-Check-Provisioning.ps1`	Microsoft Graph: the sync job Active/Enabled/Quarantined, the sync-service SP presence, `onPremisesSyncEnabled`; flags `AzureDirectoryServiceAuthorizationFailed` + deletion-threshold

Script	What it does
`CLDSYNCDX-Test-Object.ps1`	Active Directory: `IsCriticalSystemObject`, in-scope-OU test, UPN routability, and a same-UPN/proxyAddress duplicate advisory for one user

What the buyer receives

- This diagnostics toolkit as a Word document and PDF.
- All five PowerShell tools in a single ZIP.
- A repeatable diagnostic process: Collect → Analyze → Remediate → Verify, with pass/fail gates.

Outcome

The reason cloud sync is skipping users — JoinNotFound, an already-existing target, an out-of-scope OU, a critical object, a quarantined job, a missing service principal, a deletion threshold, or an unhealthy agent — isolated fast with read-only tools, so provision on demand creates the user and the provisioning log shows **action = Create**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CLDSYNCDX-Check-Agent.ps1

Purpose: Checks, with PASS/FAIL per precondition, everything the cloud sync provisioning agent needs to be healthy on this host: the "Microsoft Entra Provisioning Agent" and "Microsoft Entra Connect Agent Updater" services are Running, the PowerShell execution policy is workable (not Unrestricted), a sample outbound 443 endpoint is reachable, and an agent config / proxy is present. Prints the exact remedy for each FAIL. Read-only: it never changes anything. HOW TO RUN Run ELEVATED on the provisioning-agent host: powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Check-Agent.ps1 powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Check-Agent.ps1 -TestEndpoint login.microsoftonline.com No Microsoft Graph scopes needed — this inspects the local host only. EXPECTED RESULTS A PASS/FAIL line per precondition; each FAIL prints its remedy (start the service, open outbound 443, set the proxy, adjust the execution policy). All PASS => the agent side is healthy; move to the tenant-side checks (see 06-troubleshooting.md §Agent health). .NOTES White-label — the repeatable way. Read-only local inspection. Cloud sync service internals deferred to vendor docs. >

How to run: Run ELEVATED on the provisioning-agent host: powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Check-Agent.ps1 powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Check-Agent.ps1 -TestEndpoint login.microsoftonline.com No Microsoft Graph scopes needed — this inspects the local host only. EXPECTED RESULTS A PASS/FAIL line per precondition; each FAIL prints its remedy (start the service, open outbound 443, set the proxy, adjust the execution policy). All PASS => the agent side is healthy; move to the tenant-side checks (see 06-troubleshooting.md §Agent health). .NOTES White-label — the repeatable way. Read-only local inspection. Cloud sync service internals deferred to vendor docs. >

CLDSYNCDX-Check-Provisioning.ps1

Purpose: Read-only Microsoft Graph check of the tenant-side state behind "nothing is provisioning". Reports: the cloud sync SYNCHRONIZATION JOB and whether it is Active / Enabled / QUARANTINED (with the quarantine reason and escrow/series count); whether the "Microsoft Entra AD Synchronization Service" service principal EXISTS; and the org's onPremisesSyncEnabled flag. It flags the AzureDirectoryServiceAuthorizationFailed symptom (missing sync SP) and the object-deletion-threshold symptom. Makes NO changes (never restarts or clears quarantine). HOW TO RUN powershell -File .\CLDSYNCDX-Check-Provisioning.ps1 powershell -File .\CLDSYNCDX-Check-Provisioning.ps1 -ServicePrincipalId <cloud-sync-sp-object-id> Required Microsoft Graph scopes (delegated, read-only): Synchronization.Read.All Directory.Read.All EXPECTED RESULTS The job status (Active/Paused/Quarantine), quarantine reason if any, the sync-service SP present/absent verdict, and onPremisesSyncEnabled. Quarantine / missing SP / disabled sync point you at the matching remedy in 03-implementation.md (Phase 3) and 06-troubleshooting.md. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Cloud sync service internals deferred to vendor docs. >

How to run: powershell -File .\CLDSYNCDX-Check-Provisioning.ps1 powershell -File .\CLDSYNCDX-Check-Provisioning.ps1 -ServicePrincipalId <cloud-sync-sp-object-id> Required Microsoft Graph scopes (delegated, read-only): Synchronization.Read.All Directory.Read.All EXPECTED RESULTS The job status (Active/Paused/Quarantine), quarantine reason if any, the sync-service SP present/absent verdict, and onPremisesSyncEnabled. Quarantine / missing SP / disabled sync point you at the matching

remedy in 03-implementation.md (Phase 3) and 06-troubleshooting.md. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Cloud sync service internals deferred to vendor docs. >

CLDSYNCDX-Collect-Diagnostics.ps1

Purpose: The entry point. Read-only, on-host snapshot for a cloud sync "users skipped / JoinNotFound / not provisioning" incident. Collects: the "Microsoft Entra Provisioning Agent" and "Microsoft Entra Connect Agent Updater" service states; the PowerShell execution policy; the presence of a proxy / agent configuration; the agent Trace folder (C:\ProgramData\Microsoft\Azure AD Connect Provisioning Agent\Trace); and — if the AADCloudSyncTools module is available — a full Export-AADCloudSyncToolsLogs bundle. Everything is zipped to a timestamped output folder. Makes NO changes. HOW TO RUN Run ELEVATED on the provisioning-agent host (needed to read the services and the Trace folder): powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Collect-Diagnostics.ps1 powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Collect-Diagnostics.ps1 -OutFolder .\CloudSyncDiag No Microsoft Graph scopes needed — this inspects the local host only. EXPECTED RESULTS A console summary and a timestamped folder/zip containing the agent Trace folder, the service state, and (if the module is present) the AADCloudSyncTools log export — ready to hand to the focused Check-* / Get-* tools or to a support case (see 03-implementation.md). .NOTES White-label — the repeatable way. Read-only local inspection. Cloud sync service internals deferred to vendor docs. >

How to run: Run ELEVATED on the provisioning-agent host (needed to read the services and the Trace folder): powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Collect-Diagnostics.ps1 powershell -ExecutionPolicy Bypass -File .\CLDSYNCDX-Collect-Diagnostics.ps1 -OutFolder .\CloudSyncDiag No Microsoft Graph scopes needed — this inspects the local host only. EXPECTED RESULTS A console summary and a timestamped folder/zip containing the agent Trace folder, the service state, and (if the module is present) the AADCloudSyncTools log export — ready to hand to the focused Check-* / Get-* tools or to a support case (see 03-implementation.md). .NOTES White-label — the repeatable way. Read-only local inspection. Cloud sync service internals deferred to vendor docs. >

CLDSYNCDX-Get-ProvisioningLog.ps1

Purpose: Read-only Microsoft Graph query of the cloud sync PROVISIONING LOGS (auditLogs/provisioning) for a single object, given its Active Directory ObjectGuid (preferred — the reliable key) or its UPN. Prints the latest entries with the provisioningAction (create/update/delete/other), the status (success/skipped/failure), the skipReason (e.g. JoinNotFound), and the per-step detail (Import / Determine if in scope / Match user / Determine object action) so the operator sees the authoritative reason the object was skipped or created. Makes NO changes. HOW TO RUN powershell -File .\CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid 11111111-2222-3333-4444-555555555555 powershell -File .\CLDSYNCDX-Get-ProvisioningLog.ps1 -UserPrincipalName user@contoso.com Required Microsoft Graph scopes (delegated, read-only): AuditLog.Read.All Directory.Read.All EXPECTED RESULTS For the object: the most recent provisioning entries with action, status, and skipReason, plus each pipeline step's name/status/details. "skipped / JoinNotFound" points you at 06-troubleshooting.md (the core symptom); "Create/Update" confirms a fix (see 05-testing.md T8). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Cloud sync service internals deferred to vendor docs. >

How to run: powershell -File .\CLDSYNCDX-Get-ProvisioningLog.ps1 -ObjectGuid 11111111-2222-3333-4444-555555555555 powershell -File .\CLDSYNCDX-Get-ProvisioningLog.ps1

-UserPrincipalName user@contoso.com Required Microsoft Graph scopes (delegated, read-only): AuditLog.Read.All Directory.Read.All EXPECTED RESULTS For the object: the most recent provisioning entries with action, status, and skipReason, plus each pipeline step's name/status/details. "skipped / JoinNotFound" points you at 06-troubleshooting.md (the core symptom); "Create/Update" confirms a fix (see 05-testing.md T8). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Cloud sync service internals deferred to vendor docs. >

CLDSYNCDX-Test-Object.ps1

Purpose: Read-only Active Directory check for ONE user, to catch the "excluded / out-of-scope / already-exists" reasons behind a JoinNotFound / skipped object. Reports: IsCriticalSystemObject (excluded by design if true); whether the object's OU is UNDER one of the operator-supplied in-scope OU DNs (out of scope for the scheduled cycle if not); whether the UPN suffix is a routable domain (and, if -VerifiedDomains is supplied, whether it is one of them); and an ADVISORY on other AD objects sharing the same userPrincipalName or proxyAddresses (a possible existing/duplicate target). Makes NO changes. HOW TO RUN Run on the agent host or a domain member with the ActiveDirectory module: powershell -File .\CLDSYNCDX-Test-Object.ps1 -Identity jsmith powershell -File .\CLDSYNCDX-Test-Object.ps1 -Identity jsmith `-InScopeOU 'OU=Cloud Sync - In-Scope Users - OU,DC=contoso,DC=com'` -VerifiedDomains contoso.com,contoso.onmicrosoft.com No Microsoft Graph scopes needed — this inspects Active Directory only. EXPECTED RESULTS A PASS/FAIL/ADVISORY line per condition. Critical object, out-of-scope OU, a non-routable UPN, or a duplicate advisory each map to a cause in 06-troubleshooting.md (Scope & matching / Already-exists / the core JoinNotFound symptom). .NOTES White-label — the repeatable way. Read-only against Active Directory. Cloud sync service internals deferred to vendor docs. >

How to run: Run on the agent host or a domain member with the ActiveDirectory module: powershell -File .\CLDSYNCDX-Test-Object.ps1 -Identity jsmith powershell -File .\CLDSYNCDX-Test-Object.ps1 -Identity jsmith `-InScopeOU 'OU=Cloud Sync - In-Scope Users - OU,DC=contoso,DC=com'` -VerifiedDomains contoso.com,contoso.onmicrosoft.com No Microsoft Graph scopes needed — this inspects Active Directory only. EXPECTED RESULTS A PASS/FAIL/ADVISORY line per condition. Critical object, out-of-scope OU, a non-routable UPN, or a duplicate advisory each map to a cause in 06-troubleshooting.md (Scope & matching / Already-exists / the core JoinNotFound symptom). .NOTES White-label — the repeatable way. Read-only against Active Directory. Cloud sync service internals deferred to vendor docs. >